Information Security & Cryptography

Comprehensive Midterm Notes

Lectures 1–4 + AES + Key Distribution + OTP + Modular Arithmetic

1. Information Security Basics

1.1 Key Definitions

- **Information:** Data that has been processed, organized, or structured in a meaningful context so it can be understood and used.
- **Security:** The quality/state of being free from danger.
- **Information Security:** Protection of information and its critical elements — including systems and hardware that use, store, and transmit that information.

1.2 Layers of Security (in an organization)

1. Physical Security
2. Personal Security
3. Operations Security
4. Communications Security
5. Network Security ← (Focus of this course)
6. Information Security

1.3 CIA Triad — Three Pillars of Security

Pillar	Definition
Confidentiality	Only authorized users can access the information — keep it private.
Integrity	Information must not be modified/altered without authorization. No changes to data.
Availability	Authorized users can access information anywhere, anytime when needed.

1.4 Critical Characteristics of Information (7 Characteristics)

7. **Availability** — Information is accessible to authorized users when needed.
8. **Accuracy** — Information is correct, precise, free from errors.
9. **Authenticity** — Verifies the origin and trustworthiness of information.
10. **Confidentiality** — Protects sensitive info from unauthorized access/disclosure.
11. **Integrity** — Protects info from unauthorized alteration or corruption.
12. **Utility** — Information is useful and relevant; no garbage/extra content.
13. **Possession** — Control and ownership of information.

1.5 Subsets of Information Security

- **Cyber Security** — Protects systems, networks, and programs from digital attacks.
- **Computer Security** — Tools designed to protect data and thwart hackers.
- **Network Security** — Measures to protect data during transmission.
- **Internet Security** — Measures to protect data during transmission over interconnected networks.

1.6 CNSS Security Model (3D Model)

The CNSS (Committee on National Security Systems) model is a three-dimensional security model.

Dimension	Elements
Security Goals (Dimension 1)	Confidentiality, Integrity, Availability (CIA)
Information States (Dimension 2)	Transmission, Storage, Processing
Security Measures (Dimension 3)	Education/Training, Awareness, People, Technology, Policy

📌 **NOTE:** Different groups within an organization are responsible for each state. E.g., the network security group handles data during transmission; programmers/data entry handle data during processing.

1.7 Three Aspects of Information Security

1.7.1 Security Attacks

Any action that compromises the security of information owned by an organization.

Type	Description	Example
Passive Attack	Collects/monitors information without changing it. Damages Confidentiality.	Traffic analysis, eavesdropping, spyware/malware that collects info.
Active Attack	Changes/modifies the original message/content. Damages Integrity.	Message hijacking, replay attacks, modification of data in transit (phishing).

📌 **NOTE:** Phishing attack = Creating a same-to-same replica of a legitimate site/message to deceive users.

1.7.2 Security Services

- **Authentication** — Assurance that the communicating entity is who they claim to be.
- **Access Control** — Prevention of unauthorized use of a resource.
- **Data Confidentiality** — Protection of data from unauthorized disclosure.
- **Data Integrity** — Assurance that data received is exactly as sent by an authorized entity.
- **Non-Repudiation** — Protection against denial by one of the parties in a communication.
- **Availability of Service** — Physical security, authentication, authorization.

1.7.3 Security Mechanisms

A feature designed to detect, prevent, or recover from a security attack.

Type	Description	Examples

Specific Security Mechanism	A process designed to identify, avoid, or restore from a security attack.	Encipherment, digital signatures, access controls, data integrity mechanisms, authentication exchange, traffic padding, routing control, notarization.
Pervasive Security Mechanism	Cannot be specific to a particular service; general security measures.	Trusted functionality, security labels, event detection, security audit trails, security recovery.

2. Cryptography Basics

2.1 What is Cryptography?

Cryptography (from Greek: "hidden writing") is the practice of transforming information so that it is secure and cannot be accessed by unauthorized parties — by scrambling the message so only approved recipients can read it.

- **Steganography** — Hides the existence of a message (e.g., hidden in an image). Cryptography scrambles it; steganography hides it.

2.2 Basic Terminology

Term	Definition
Plaintext	The original, readable message or data before encryption.
Ciphertext	The scrambled, encrypted message after applying the algorithm.
Encryption / Enciphering	The process of converting plaintext → ciphertext.
Decryption / Deciphering	The process of converting ciphertext → plaintext.
Cryptography	The study of encryption schemes (writing secret codes).
Cryptanalysis	The study of breaking ciphers / reading encrypted messages without the key.
Cryptology	The combined discipline of cryptography + cryptanalysis.
Key	A value (independent of plaintext) that controls the encryption/decryption algorithm.

2.3 Symmetric vs. Asymmetric Encryption

Type	Key Used	How it works	Example
Symmetric	Same key for encryption & decryption	Sender & receiver share one secret key. Fast, but key distribution is a problem.	AES, DES
Asymmetric	Two keys: Public key (encrypt) +	Anyone can encrypt with public key; only owner's private key decrypts it.	RSA, Diffie-Hellman

	Private key (decrypt)		

2.4 Symmetric Cipher Model — 5 Ingredients

14. **Plaintext** — Original input message.
15. **Encryption Algorithm** — Performs substitutions and transformations on plaintext.
16. **Secret Key** — Input to the algorithm; independent of plaintext; controls the substitutions/transformations.
17. **Ciphertext** — The scrambled output — appears random.
18. **Decryption Algorithm** — The reverse algorithm; uses the same key to recover plaintext.

```
Plaintext → [Encryption Algo + Key] → Ciphertext → [Decryption Algo +
                                Key] → Plaintext
```

2.5 Cryptanalysis and Brute-Force Attacks

- **Cryptanalysis** — Relies on the nature of the algorithm + knowledge of plaintext characteristics (e.g., typing style, language frequency). Goal: decrypt a message or recover the key.
- **Brute-Force Attack** — Tries every possible key until the correct one is found. Always possible; proportional to key size.

Types of Cryptanalytic Attacks

Attack Type	Information Available to Attacker
Ciphertext Only	Only the encrypted message — hardest for attacker.
Known Plaintext	Attacker has some plaintext-ciphertext pairs.
Chosen Plaintext	Attacker can choose plaintext and get its encryption.
Chosen Ciphertext	Attacker can choose ciphertext and get its decryption.
Chosen Text	Combination of chosen plaintext + ciphertext.

- **Unconditionally Secure:** Ciphertext doesn't contain enough info to determine plaintext — no matter how much time the attacker has. (OTP is the only known scheme.)
- **Computationally Secure:** The cost/time to break exceeds the value of the information, or the time to break exceeds the useful lifetime of the info.

3. Types of Ciphers

3.1 Two Main Traditional Transformations

- **Substitution Cipher** — Replaces specific data sequences with other data sequences (letters replaced with different letters).
- **Transposition Cipher** — Rearranges the positions/order of letters without changing the letters themselves (same letters, new order).

3.2 Caesar Cipher (Substitution)

The Roman emperor Julius Caesar substituted each letter with the letter 3 positions further in the alphabet.

```
Encryption: C = (P + K) mod 26
Decryption: P = (C - K) mod 26
```

Where: P = Plaintext value (A=0, B=1,...Z=25), K = Key/shift value, C = Ciphertext value

Example: Encrypt "HOTEL" with K=7

Letter	Value (P)	P + 7	(P+7) mod 26	Ciphertext Letter
H	7	14	14	O
O	14	21	21	V
T	19	26	0 (wraps to 0)	A
E	4	11	11	L
L	11	18	18	S

📌 **NOTE:** HOTEL → OVALS with K=7. When the result = 26, it wraps back to 0 (like a clock).

Original Caesar (K=3)

- p: ABCDEFGHIJKLMNOPQRSTUVWXYZ
- F(p): DEFGHIJKLMNOPQRSTUVWXYZABC
- Example: ET TU BRUTUS → HW WX EUXWXV

3.3 ROT-13 Cipher

Each letter is substituted with the letter 13 positions away. Applying ROT-13 twice gives back the original.

- p: ABCDEFGHIJKLM | NOPQRSTUVWXYZ
- F(p): NOPQRSTUVWXYZ | ABCDEFGHIJKLM
- Example: THIS IS A SECRET → GUVF VF N FRPERG

3.4 Affine Cipher (Substitution)

A mono-alphabetic substitution cipher. Uses TWO keys (a and b) and a multiplication step — more secure than Caesar.

```
Encryption: C = (a × P + b) mod 26    [E(x) = (ax + b) mod m]
Decryption: P = a⁻¹ × (C - b) mod 26  [D(x) = a⁻¹(x - b) mod m]
```

📌 **NOTE:** 'a' and m (26) must be COPRIME — they share no common factors other than 1. Valid values of a: 1, 3, 5, 7, 9, 11, 15, 17, 19, 21, 23, 25.

Example: Encrypt "HI" with a=5, b=8

Letter	P value	Calculation	Result	Ciphertext
H	7	(5×7 + 8) mod 26 = 43 mod 26	17	R

I	8	(5×8 + 8) mod 26 = 48 mod 26	22	W

Result: "HI" → "RW"

How to Calculate mod 26

19. Divide the number by 26
20. Drop the whole number part (before decimal)
21. Multiply the decimal part × 26 → this is your remainder

- Example: 210 mod 26 → 210 ÷ 26 = 8.076... → 0.076 × 26 = 2 ∴ result = 2

Affine Decryption — Finding Modular Inverse

For Affine cipher with a=5, b=8, find 5^{-1} mod 26:

- Need a number such that: $5 \times a^{-1} \equiv 1 \pmod{26}$
- Testing: 5 × 21 = 105; 105 mod 26 = 1 ✓
- So: $5^{-1} = 21 \pmod{26}$
- Decryption formula becomes: D(y) = 21(y - 8) mod 26

3.5 XOR Cipher (from class notes)

Performs XOR on ASCII binary values of each character using a repeating key.

```
XOR Rule: Same bits (0⊕0 or 1⊕1) → 0   |   Different bits (0⊕1 or 1⊕0)
                                   → 1
 Property: A ⊕ B ⊕ B = A  (applying XOR twice with same key restores
                                 original)
```

Example (from class notes): Encrypt "SECURITY" with key "KEYKEYKE"

Plaintext	ASCII	Key	ASCII	Binary PT	Binary Key	XOR Result	Decimal
S	83	K	75	01010011	01001011	00011000	24
E	69	E	69	01000101	01000101	00000000	00
C	67	Y	89	01000011	01011001	00011010	26
U	85	K	75	01010101	01001011	00011110	30
R	82	E	69	01010010	01000101	00010111	23
I	73	Y	89	01001001	01011001	00010000	16
T	84	K	75	01010100	01001011	00011111	31
Y	89	E	69	01011001	01000101	00011100	28

Cipher result (decimal): 24 00 26 30 23 16 31 28

3.6 Transposition Ciphers

Letters are REARRANGED — same letters, new order. No substitution.

3.6.1 Rail Fence Cipher

Plaintext is written diagonally in a zig-zag pattern across multiple "rails", then read row by row.

- Key = number of rails
- Example (key=2): "meet me after the toga party"

Rail 1: m e m a t r h t g p r y
Rail 2: e t e f e t e o a a t

- Ciphertext: MEMATRHTGPRYETEFETEOAAT

3.6.2 Columnar Transposition Cipher

Plaintext written in rows under a keyword. Columns are read in alphabetical order of the key letters.

- Step 1: Number key letters alphabetically
- Step 2: Write plaintext in rows under the key
- Step 3: Read columns in alphabetical order (1→5)

Example: Plaintext = "WE ARE DISCOVERED" | Key = ZEBRA

Z(5)	E(3)	B(2)	R(4)	A(1)
W	E	A	R	E
D	I	S	C	O
V	E	R	E	D

Read columns in order 1→5: Col A(1)=EOD, Col B(2)=ASR, Col E(3)=EIE, Col R(4)=RCE, Col Z(5)=WDV

- Ciphertext: EODASREIERCEWDV

Another example: Plaintext = TRANSPOSITIONCIPHER | Key = SECRET

- Ciphertext: AIIXRSCXSIHXNTPXTONRPOEX

3.6.3 Types of Transposition Ciphers

Type	Description
Rail Fence	Letters written in zig-zag across N rails; read row by row.
Columnar Transposition	Written in rows under keyword; columns read in alphabetical key order.
Double Transposition	Applies columnar transposition TWICE for added security.

4. Modular Arithmetic in Cryptography

4.1 What is Modular Arithmetic?

Called "clock arithmetic" — numbers wrap around after reaching a certain value (the modulus). It is the backbone of most encryption algorithms.

```
a mod n = remainder when a is divided by n
```

4.2 Modular Exponentiation

Computing large powers modulo a number efficiently. Used in RSA, Diffie-Hellman.
Method: Square-and-reduce. Keep numbers small by reducing mod n at each step.

Example: Compute 18^{13} mod 39

22. 18^2 mod 39 = 324 mod 39 = 12
23. 18^4 mod 39 = 12^2 mod 39 = 144 mod 39 = 27
24. 18^8 mod 39 = 27^2 mod 39 = 729 mod 39 = 27
25. Break exponent: 13 = 8 + 4 + 1 → $18^{13} = 18^8 \times 18^4 \times 18^1$ mod 39
26. = 27 × 27 × 18 mod 39 = 729 mod 39 × 18 = 27 × 18 = 486 mod 39 = 18

$$18^{13} \bmod 39 = 18$$

Example: Compute 5^{12} mod 13

27. 5^2 mod 13 = 25 mod 13 = 12
28. 5^4 mod 13 = 12^2 mod 13 = 144 mod 13 = 1
29. $5^{12} = (5^4)^3 = 1^3 = 1$

$$5^{12} \bmod 13 = 1$$

Example: Compute 7^{10} mod 15

30. 7^2 mod 15 = 49 mod 15 = 4
31. 7^4 mod 15 = 4^2 mod 15 = 16 mod 15 = 1
32. $7^{10} = (7^4)^2 \times 7^2$ mod 15 = $1^2 \times 4 = 4$

$$7^{10} \bmod 15 = 4$$

5. One-Time Pad (OTP)

5.1 Definition

OTP combines plaintext with a completely random key (the pad) using XOR. It is the ONLY theoretically unbreakable encryption — but only under strict conditions.

5.2 Four Required Conditions (OTP is secure only if ALL four are met)

33. **Random Key** — The pad must be completely random.
34. **Same Length** — Key must be exactly the same length as the message.
35. **Used Once** — The key must NEVER be reused. Reusing even once destroys security.
36. **Shared Secret** — Both sender and receiver must have the exact same key.

📌 **NOTE:** If the key is wrong even by ONE bit, the decrypted output becomes completely incorrect.

5.3 OTP Encryption Example: Plaintext = "HELLO", Key = "XMCKL"

Step 1: Convert to Binary (ASCII)

Letter	ASCII Binary
H	01001000
E	01000101
L	01001100
L	01001100
O	01001111
Key Letter	**ASCII Binary**
X	01011000
M	01001101
C	01000011
K	01001011
L	01001100

Step 2: Apply XOR

Plain	Binary	Key	Binary	XOR Result	Decimal
H	01001000	X	01011000	00010000	16
E	01000101	M	01001101	00001000	8
L	01001100	C	01000011	00001111	15
L	01001100	K	01001011	00000111	7
O	01001111	L	01001100	00000011	3

Step 3: Ciphertext

The resulting decimal values (16, 8, 15, 7, 3) map to non-printable ASCII control characters (Data Link Escape, Backspace, Shift In, Bell, End of Text). This is normal — OTP output is raw binary data.

5.4 OTP Decryption

```
Plaintext ⊕ Key = Ciphertext
Ciphertext ⊕ Key = Plaintext   (XOR with same key reverses the operation)
```

Apply XOR again with SAME key XMCKL to recover HELLO:

Cipher Binary	Key (Binary)	XOR Result	Letter
00010000	X: 01011000	01001000	H
00001000	M: 01001101	01000101	E
00001111	C: 01000011	01001100	L
00000111	K: 01001011	01001100	L
00000011	L: 01001100	01001111	O

5.5 Why OTP is Secure / Why OTP is Impractical

Secure Because	Impractical Because
Each letter encrypted with truly random key	Key must be as long as the message
No pattern to analyze	Key must be shared securely beforehand
Cannot be broken even with unlimited computing power	Key must NEVER be reused

6. AES — Advanced Encryption Standard

6.1 Introduction

- **AES** = Advanced Encryption Standard. Made in 2001 by NIST (USA), replacing the old DES algorithm.
- **Type:** Symmetric block cipher (same key for encryption and decryption).
- **Algorithm:** Rijndael algorithm.
- **Block size:** 128 bits (16 bytes = 4×4 state matrix).
- **Key sizes:** 128, 192, or 256 bits.

Key Length	AES Name	Number of Rounds
128 bits	AES-128	10
192 bits	AES-192	12
256 bits	AES-256	14

6.2 Why AES is Strong

- **AES-128** requires trying 2^{128} combinations — practically impossible to brute-force.
- Resistant to known attacks: Brute-Force, Statistical, Differential, and Linear Attacks.
- Works very fast on both software and hardware.
- Used in: banks, Wi-Fi security (WPA2), VPNs, WhatsApp, and almost every secure app.

6.3 AES Structure — Input/State Array

AES operates on a 4×4 matrix of bytes called the State Array.

- 1 word = 4 bytes = 32 bits
- Block size = 128 bits = 16 bytes = 4 words = 4×4 matrix

6.4 Four AES Transformation Functions (applied each round)

Step	Transformation	Description
1	Substitute Bytes (SubBytes)	Replace each byte using a 16×16 S-box table. Byte {95} → look at row 9, col 5 → value {2A}.

2	Shift Rows (ShiftRows)	Circular byte shift of each row. Row 1 unchanged. Row 2: 1-byte shift left. Row 3: 2-byte shift left. Row 4: 3-byte shift left.
3	Mix Columns (MixColumns)	Each column processed separately. Each byte replaced by a value dependent on all 4 bytes in its column. (Matrix multiplication in GF(2^8))
4	Add Round Key (AddRoundKey)	XOR the state with 128-bit round key. Simplest step — designed to be as simple as possible.

6.5 AES Round Structure

Before rounds begin: Initial AddRoundKey (XOR plaintext with Key 0).
Then each round applies: SubBytes → ShiftRows → MixColumns → AddRoundKey.
Final round: SubBytes → ShiftRows → AddRoundKey (no MixColumns in last round).

6.6 AES Solved Example (from slides): Plaintext = "ok" (6F 6B), Key = A7 3B

Step 1: AddRoundKey (XOR plaintext with key)

Value	Hex	Binary
Plaintext	6F 6B	0110 1111 0110 1011
Key	A7 3B	1010 0111 0011 1011
XOR Result	C8 50	1100 1000 0101 0000

Step 2: SubBytes — look up each byte in S-box

Step 3: ShiftRows — circular shift of rows

Step 4: MixColumns — matrix multiplication

Step 5: AddRoundKey with Round Key 1

6.7 AES-128 Detailed Example (from AES_round_0.docx)

Plaintext (Hex): 32 43 f6 a8 88 5a 30 8d 31 31 98 a2 e0 37 07 34
Key 0 (Hex): 2b 7e 15 16 28 ae d2 a6 ab f7 15 88 09 cf 4f 3c

Step 0: Initial AddRoundKey (XOR each byte)

Byte #	Plaintext	Key 0	Result (XOR)
1	32	2b	19
2	43	7e	3d
3	f6	15	e3
4	a8	16	be
5	88	28	a0
6	5a	ae	f4
7	30	d2	e2
8	8d	a6	2b

Key Schedule (Round Key Generation)

For AES-128, the key expansion generates 11 round keys (Round Key 0 through Round Key 10).

The G function is applied on the last word (W3) of previous key:

37. Left-shift W3 by 1 byte
38. Apply S-box substitution
39. XOR with round constant (Rcon)

Then: $W4 = W0 \oplus G(W3)$, $W5 = W1 \oplus W4$, $W6 = W2 \oplus W5$, $W7 = W3 \oplus W6$

7. Key Distribution & Management

7.1 Overview

Encryption is only as strong as the keys that power it. Key management involves: Generation → Distribution → Storage → Rotation → Revocation.

7.2 Placement of Encryption

Type	Description	Used In
Link-to-Link Encryption	Encrypts data between two specific points. Data is decrypted and re-encrypted at each intermediate node. Like a relay race of couriers.	HTTPS, WPA2/WPA3, VPNs
End-to-End Encryption (E2EE)	Encrypts from sender to receiver — no intermediate node can read it. Like a locked box only sender & receiver can open.	WhatsApp, Signal, iMessage, ProtonMail

📌 **NOTE:** Link encryption weakness: If the intermediate server/node is compromised, data can be exposed after decryption at that node.

7.3 Key Distribution for Link Encryption (Symmetric)

Uses symmetric keys. The key distribution problem: how to securely share the same key?

- **Diffie-Hellman Key Exchange** — Used when browsers connect via HTTPS. Agrees on a temporary symmetric key WITHOUT transmitting the key over the network.
- **Session Keys** — Temporary keys discarded after the session ends. Past sessions remain safe even if later compromised.

7.4 Key Distribution for E2EE (Asymmetric + Symmetric)

- **Public Key** = like a mailbox slot — anyone can insert (encrypt). Only owner has private key to open (decrypt).
- **Asymmetric + Symmetric Hybrid:** Use public key to encrypt a temporary symmetric key → then use that symmetric key to communicate.
- **PGP/GPG** = Pretty Good Privacy / GNU Privacy Guard. Uses public-key cryptography to exchange symmetric keys for email encryption.

7.5 Key Management Steps

40. **Key Generation** — Use secure algorithms (AES for symmetric, RSA for asymmetric).
41. **Key Distribution** — Share keys securely (e.g., banks distribute ATM keys via dedicated secure channels).
42. **Key Storage** — Use Hardware Security Modules (HSM) or encrypted databases.
43. **Key Rotation** — Periodically replace old keys (e.g., every 24 hours). Like regularly changing your door lock.
44. **Key Revocation** — Remove/invalidate compromised or expired keys.

7.6 Diffie-Hellman Key Exchange

Introduced by Whitfield Diffie and Martin Hellman in 1976. Allows two parties to establish a shared secret key over an insecure channel without meeting in person.

Step-by-Step Example

Step	Action	Values (example)
1	Agree on public parameters (shared openly — even if attacker knows these, it's safe)	n = 5, g = 7 (n and g are large prime numbers)
2	Each party picks a private number (NEVER shared)	Mr. A: x = 3, Mr. B: y = 13
3	Each computes a public value using: $A = g^x \bmod n$ and $B = g^y \bmod n$	$A = 7^3 \bmod 5 = 343 \bmod 5 = 3$ | $B = 7^{13} \bmod 5 = 2$
4	Exchange A and B publicly (attacker can see these but cannot reverse them)	Mr. A gets B=2, Mr. B gets A=3
5	Each computes the shared key: $K = B^x \bmod n$ or $K = A^y \bmod n$	Mr. A: $K = 2^3 \bmod 5 = 8 \bmod 5 = 3$ | Mr. B: $K = 3^{13} \bmod 5 = 3$

Both get K = 3 → This is the shared secret key used for symmetric encryption!

📌 **NOTE:** Even if an attacker knows n, g, A, and B, they cannot calculate x or y (Discrete Logarithm Problem). This is what makes Diffie-Hellman secure.

8. Frequency Analysis & Cipher Security

8.1 Frequency Analysis

The statistical distribution of letter frequencies in any message tends toward the known frequency distribution of that language. This is true especially for long messages.

- Simple substitution ciphers preserve letter frequencies → can be broken by frequency analysis.
- In English: E, T, A, O, I, N are the most frequent letters.

- Discovered by Arab scientist Abu Yusuf Yaqub ibn Ishaq al-Kindi (~850 CE).
- Simple substitution cipher has 26! ≈ 4×10^{26} possible keys — exhaustive search is not practical, but frequency analysis can crack it.

8.2 Summary: Cipher Security Comparison

Cipher	Security Level	Key Space	Attack Vulnerability
Caesar	Very Weak	Only 26 keys	Brute force (trivial)
Simple Substitution	Weak	26! ≈ 4×10^{26}	Frequency analysis
ROT-13	Very Weak	1 key (fixed)	Trivial
Affine	Weak-Medium	Limited by coprime 'a'	Frequency analysis
OTP	Unbreakable	Key = message length	Theoretically unbreakable if used correctly
AES-128	Very Strong	2^{128}	No practical attack known

Quick Reference — Formulas & Key Facts

Topic	Formula / Key Fact
Caesar Encrypt	C = (P + K) mod 26
Caesar Decrypt	P = (C - K) mod 26
Affine Encrypt	C = (a × P + b) mod 26
Affine Decrypt	P = a^{-1} × (C - b) mod 26 (a and 26 must be coprime)
OTP Encrypt	Ciphertext = Plaintext ⊕ Key
OTP Decrypt	Plaintext = Ciphertext ⊕ Key (XOR property: A⊕B⊕B = A)
AES Block Size	128 bits = 16 bytes = 4×4 matrix
AES Keys	128 / 192 / 256 bits → 10 / 12 / 14 rounds
AES Transformations	SubBytes → ShiftRows → MixColumns → AddRoundKey (each round)
Diffie-Hellman Public Values	A = g^x mod n (Mr. A) | B = g^y mod n (Mr. B)
Diffie-Hellman Shared Key	K = B^x mod n = A^y mod n
Modular Exponentiation	Square-and-reduce method: compute powers of 2, then combine
CIA Triad	Confidentiality, Integrity, Availability
CNSS Dimensions	Goals (CIA) × States (Transmission/Storage/Processing) × Measures (People/Tech/Policy)
Brute Force	Try every possible key; complexity = 2^(key size)

Good luck on your midterm exam!